# CS4455 Cybersecurity — Epic Project 2026

| | |
|---|---|
| **Module** | CS4455 Cybersecurity |
| **Programme** | Immersive Software Engineering (ISE), 2nd Year |
| **Version** | 1.0 (DRAFT — for instructor review) |
| **Date** | 18th May 2026 |
| **Block Leader** | Mark Burkley |

## Instructors

| **Major** | **Instructor** |
|---|---|
| Computer Networks & Cybersecurity | Mark Burkley |
| C++ Programming | Kashif Memon |
| Cryptography | Eoin O'Brien |
| Blockchain | Andrew Le Gear |

## Project Summary

In this project, students are tasked with the design and implementation of a **secure messaging application** that guarantees confidentiality, integrity, and authenticity of communications. The application must incorporate concepts from all four subjects studied during the CS4455 block.

Students will build one or more desktop clients that connect to a common back-end server. At a minimum, a client must be created that uses C++ should use appropriate libraries to do cryptography and secure connectivity.

A second optional client can be created using HTML and JavaScript and should use the Web Crypto API and web security elements. Clients should support operations such as the following:

- User sign-up, login and password management
- Viewing a list of sent and received messages
- Creating and sending a new message
- Forwarding a message to another user after verifying their identity
- Revoking a user's access to a previously shared message
- Downloading a message (owned or shared)
- Deleting a message

Students will build a server application to handle authentication and to provide an API for the clients to interact with. The back-end can be developed in any language. NodeJS, Python, or other can be used.

The messaging system must employ end-to-end encryption, secure network connectivity, and a C++ client component. A blockchain element will record message digests and timestamps to provide tamper-evident integrity verification.

Teams have flexibility in their messaging architecture (real-time, asynchronous, or hybrid) but must satisfy the requirements of each minor as described below.

**Overall Marking Scheme:** The CS4436 block is divided into four equally weighted subjects. Each subject contributes 25% to the overall epic project grade, for a total of 100%. Within each subject, marks are distributed across the criteria defined in that subject's rubric.

## Key Dates

| Milestone | Date |
|---|---|
| Brief released to students | Monday 18th May 2026 |
| Weekly status report 1 | Friday 23rd May 2026, 5:00 PM |
| Weekly status report 2 | Friday 30th May 2026, 5:00 PM |

| Milestone | Date |
|---|---|
| **Project submission deadline** | **Wednesday 3rd June 2026, 5:00 PM** |
| Presentations & interviews | 4th and 5th June 2026 |

## Marks Breakdown

| Minor | Weight |
|---|---|
| Computer Networks & Cybersecurity (Burkley) | 25% |
| C++ Programming (Memon) | 25% |
| Cryptography (O'Brien) | 25% |
| Blockchain (Le Gear) | 25% |
| **Total** | **100%** |

## Students

| Name | ID | Group |
|---|---|---|
| Sarah McDonagh | 24403067 | sas |
| Sreejita Saha | 24412465 | sas |
| Aoibheann Mangan | 24421928 | sas |
| SkyeNova Fitzpatrick | 24402095 | 1bit2qbit |
| James Hayes | 24402796 | 1bit2qbit |
| Aaron ODoherty | 24424048 | 1bit2qbit |
| Holly Best | 24411566 | Cryptmunks |
| Matthew Burke | 24416827 | Cryptmunks |
| Julia Hooper | 24430706 | Cryptmunks |
| Emer OConnor | 24404977 | Skytree |
| Tom Byrne | 24414662 | Skytree |
| Patrick HoadeSyms | 24428469 | Skytree |
| Yasmin Woodlock | 22336877 | DES-perate |

| Name | ID | Group |
|---|---|---|
| RonanLiam Gilhool | 24402702 | DES-perate |
| Caitlin Moloney | 24411086 | DES-perate |
| Rafael Okafor | 24425249 | Zebra |
| Harry Kikkers | 24431125 | Zebra |
| Aaron McGuinness | 23355409 | Drop-table |
| Waleed Ahmad | 24255122 | Drop-table |
| MarkJames Drohan | 24403342 | Drop-table |
| Sam McLoughlin | 24403822 | hangover |
| Daniel Quinn | 24422622 | hangover |
| Andy Cao | 24422703 | hangover |
| Emmett Macken | 24403156 | Eternal-blue |
| Nathan Dobbyn | 24424609 | Eternal-blue |
| Conor Lydon | 24429635 | Eternal-blue |
| Cian McNamara | 24411256 | kfc |
| Daniel Tyutyunkov | 24417173 | kfc |
| Eoin OKelly | 24417491 | kfc |
| PatrickDominick OShea | 24430668 | upDaKingdom |
| FionnTomas OMurchu | 24405744 | upDaKingdom |
| Michael Fennelly | 24437522 | upDaKingdom |

## Project Server

A cloud server is available for teams to run cloud backends. The domain name for the project is **THEBURKENATOR.COM**. Each team will have their own virtual host (e.g. `SAS.THEBURKENATOR.COM`) and their own virtual machine that is accessible via `ALDERAAN.SOFTWARE-ENGINEERING.IE`. VMs will be created with Ubuntu Linux by default but other distros can be used as needed. Teams may install their own development environments and back-ends on their respective VMs.

## Submission Requirements

Upload a zipped archive of your project's GitHub repository to Brightspace by **Wednesday 3rd June 2026 at 5:00 PM**. This archive must include:

- All source code for the project
- A `README` file with clear instructions on how to install dependencies, set up the project, and run it

Your submission must also include a **cover document** (PDF or Markdown) alongside the zipped archive containing:

- Group name and project URL
- Full name and student ID of each group member
- URL of the GitHub repository used for the project
- A breakdown of each member's contributions, including:
    - An estimated percentage of the overall work completed by each person
    - The specific features, components, or tasks each member worked on
- Any additional design summaries, diagrams, or explanations requested by the topic-specific requirements below

### AI Prompt Artefacts (New for 2026)

Your submission **must include** a record of AI tool usage during development. This should include:

- Screenshots or exported logs of significant prompts and responses from AI coding assistants (e.g. GitHub Copilot, ChatGPT, Claude)
- A brief reflective commentary on how AI tools were used, what worked well, and what required manual correction
- Evidence of critical evaluation of AI-generated code (e.g. where you rejected, modified, or debugged AI output)

These artefacts will be discussed during the interview. Students must be able to explain their prompting strategies and critically evaluate the AI-generated code in their submission.

## Presentation and Interview

Teams will present and demonstrate their project followed by a panel interview. During the interview, students will be expected to clearly explain, justify, and defend the design decisions made in their submission across all five minors. Students must demonstrate a clear understanding of what they have implemented and submitted.

**Format:**

- Maximum **10 minutes** for team presentation and demonstration
- Maximum **20 minutes** for panel questioning (including AI prompt critique)
- Total: **30 minutes** per team (strictly enforced)

**AI Prompt Critique:**

During the interview, each student may be asked to:

- Walk through a specific AI interaction from their submitted artefacts
- Explain why they prompted in a particular way
- Identify strengths and weaknesses in the AI-generated output
- Describe what they changed and why

**Important:**

- Every student is expected to understand all the code in the project, even if they did not author it themselves
- Failure to adequately explain the solution, regardless of whether it is technically correct, will result in a loss of marks
- Marks are awarded on an **individual basis** (not every member of the team will necessarily receive the same grade)

## Subject Requirements

Each instructor has defined the requirements for their portion of the project below. Students must satisfy all four sections.

## Computer Networks & Cybersecurity (Mark Burkley) — 25%

- Secure connectivity (SSL/TLS between client and server)
    - Client verifies authenticity and validity of SSL certificate
- Server-side security and authentication
    - Users are securely authenticated and authorised
- Vulnerability testing and penetration testing report
- Network architecture documentation
    - Connections to external services (MySQL server, etc) are documented
- Front end programs should create SSL protected connections to the virtual host.
    - A back-end service running on the server should accept requests and process them.
    - Part of the back-end may be written in C++ with html parsing in NodeJS or python or the entire back-end can be written in one of those languages.
- Teams should demonstrate their ability to write code that resolves host names and establishes secure connections.
    - Using low level socket calls with libcrypto and libssl libraries would be impressive but if time is tight then using a library such as libcurl is acceptable.
- The implemented solution must be tested and ensure protection against undesired side effects.
- Students must demonstrate that they have implemented controls and have actively checked for issues such as:
    - Improper Input Validation
    - Broken Authentication
    - Broken Access Control
    - Cryptographic Issues
    - Injection
    - Security Misconfiguration
    - Sensitive Data Exposure
    - Vulnerable Components
- Both front end and backend services are expected to be resilient against common vulnerabilities and exploits.
- Students are encouraged to include a penetration testing report in their submission detailing penetration tests executed and their findings.

## C++ Programming (Kashif Memon) — 25%

Students must create a C++ component for the secure messaging application. This could be a command-line client, a small GUI/tooling component, a message-processing module, a local message store, or another C++ part that connects clearly to the EPIC project.

The C++ work does not need to implement the entire messaging system, but it must be a meaningful part of it and must be demonstrated during the final presentation/interview.

**Requirements**

**1. C++ Component**

- Build a working C++ component related to the secure messaging app.
- Examples include:
    - sending or preparing messages;
    - receiving, parsing, or displaying messages;
    - storing local message history;
    - validating message data;
    - interacting with another part of the system;
    - providing a simple client interface.

**2. Code Structure**

- Use more than one source file where appropriate.
- Use clear .h / .hpp and .cpp files.
- Provide simple build instructions.

- Use CMake if possible, or clearly document any alternative build method.

**3. Functions and Classes**

- Use functions to break the program into smaller tasks.
- Use classes to model important parts of the system, such as:
    - User
    - Message
    - Conversation
    - Client
    - MessageStore
- Use public and private access correctly.
- Use constructors where needed.

**4. Object-Oriented Programming**

- Use object-oriented programming where it helps the design.
- Use inheritance or polymorphism only where it makes sense.
- Students should be able to explain why they used their class structure.

**5. Memory Management**

- Avoid memory leaks and unsafe pointer use.
- Prefer normal objects, references, STL containers, and smart pointers.
- Use std::unique_ptr or std::shared_ptr only where appropriate.

- Students should be able to explain who owns important objects.

**6. STL and Modern C++**

- Use suitable STL containers such as:
    - std::vector
    - std::map
    - std::set
    - std::unordered_map
- Use STL algorithms where appropriate, such as:
    - std::find
    - std::sort
    - std::count
    - std::copy
- Use lambdas where useful.
- Use const and references correctly where possible.

**7. Documentation and Interview**

- Include README instructions for building and running the C++ component.
- Each student must be able to explain:
    - what the C++ component does;
    - how the code is organised;

- how the classes work;
- how memory is managed;
- what STL containers/algorithms were used;
- how AI tools were used, if applicable.

## Cryptography (Eoin O'Brien) — 25%

The messaging application must provide end-to-end encrypted (E2EE) communication between users. When Alice sends a message to Bob, only Bob can read it, and Bob can verify it genuinely came from Alice. The server relays ciphertext and stores metadata, but it cannot see message contents or undetectably tamper with them.

Your design must guarantee confidentiality, integrity, and authenticity of messages against:

- A **passive network attacker** who can read all traffic between clients and the server
- An **active network attacker** who can additionally modify, drop, replay, or inject traffic
- An **honest-but-curious server** that faithfully runs your protocol but logs everything it sees
- A **fully compromised server** controlled by the attacker, with full access to the database and able to send arbitrary responses to clients

A compromised server can drop messages, refuse to deliver them, or serve malicious public keys to new users. It **must not** be able to read message plaintexts, forge messages from one user to another, or tamper with ciphertexts without detection. Your design document **must** state explicitly which properties survive server compromise and which do not.

Beyond the wire, users authenticate to the server with passwords, and may store long-term private keys locally. Your design must protect both: passwords **must not** be recoverable from a server database breach, and local private keys at rest **must not** be recoverable from a stolen unlocked laptop.

The requirements below specify the building blocks. You are expected to compose them into a coherent design and justify the composition in your design document.

1. **End-to-End Authenticated Encryption**

a. Encrypt all message payloads under a standardised AEAD scheme between sender and recipient (e.g. AES-256-GCM, ChaCha20-Poly1305, or another standard AEAD with explicit justification).
b. The server must not be able to read plaintext or alter ciphertext undetectably, even if fully compromised; this must be demonstrable from the server's database and/or logs at the demo.
c. Custom AEAD constructions, Encrypt-and-MAC, MAC-then-Encrypt, and non-AEAD schemes are not acceptable

2. **Key Establishment and Sender Authentication**
   a. Establish shared cryptographic state between users without revealing it to the server (e.g. HPKE Mode_Auth, RFC 9180, with DHKEM(X25519, HKDF-SHA256) and an approved AEAD)
   b. Recipients must be able to verify message origin
   c. Where public keys are used, the design document must state the trust model and justify it. TOFU (with pinning) is acceptable and expected for most teams; teams opting to use PKI must justify the trust model (CA or web-of-trust) and the key revocation story.
3. **Password and Key Derivation**
   a. Server-side password verification must use an appropriate password hashing function. The choice of function and parameters must be justified.
   b. Use HKDF with explicit info and salt for any derivation of multiple keys from a shared secret.
   c. If a user's long-term private key is stored locally, encrypt it at rest under a key derived from a user secret, with KDF parameters separate from server-side password verification.
4. **Implementation Standards**
   a. Use only vetted cryptographic libraries and tools, e.g. libsodium, cryptography, PyCryptodome, Web Crypto, OpenSSL EVP, hpke-js, or pyhpke.
   b. All randomness must come from an appropriate CSPRNG.
   c. Forbidden: hand-rolled primitives, MD5 or SHA-1 in security-relevant roles, DES, 3DES, RC4, ECB mode, Dual_EC_DRBG, textbook RSA, hardcoded keys, hardcoded IVs, nonce reuse.
5. **Cryptographic Design Document (approx. 2-6 pages, PDF or Markdown)**
   a. State the threat model, identifying which security properties hold against (a) a passive network attacker, (b) an active network attacker, (c) an honest-but-curious server, and (d) a fully compromised server; properties not held in case (d) must be named explicitly.
   b. Provide a construction walkthrough with diagrams covering registration, key publication, send, receive, and storage at rest.

c. Justify every cryptographic primitive at parameter level: algorithm, parameters, security property relied upon, and why those parameters are appropriate for the deployment; "it's standard" and "Eoin recommended it" are not justifications.
d. Cite specifications (RFC, paper, whitepaper) for any protocol drawn on or simplified, with section numbers and explicit identification of what is retained, simplified, or omitted (e.g. AES-GCM parameter justification).
e. State known limitations honestly (what the design does not protect against).

6. **Understanding and Explanation**
    a. Students must be able to explain any and all aspects of the cryptographic design and implementation, including (but not limited to):
        i. AEAD and why authenticated encryption is required;
        ii. nonce handling and the consequences of nonce reuse;
        iii. the role of HKDF and domain separation;
        iv. memory-hard password hashing and parameter selection;
        v. the threat model the design defends against and the properties it does not provide; and
        vi. any deviation from recommended primitives and/or parameters.
    b. All students must be able to adequately explain and defend their cryptographic design. "I didn't do the implementation" is not an excuse.

## Blockchain (Andrew Le Gear) — 25%

Students must demonstrate the application of blockchain technology to provide tamper-evident integrity verification of messaging data.

**Requirements:**

1. **Smart Contract Development**
    - Write a Solidity smart contract that stores message conversation digest hashes periodically (keccak256).
    - Deploy the contract to the **Ethereum Sepolia testnet**
    - The contract should accept a message conversation hash and record it alongside the block timestamp
    - Provide the deployed contract address and ABI in your submission
2. **Message Digest Recording**

- When a message (or conversation segment) is sent, compute its keccak256 hash
- Write the hash to the deployed smart contract via a transaction
- Store the corresponding transaction hash for later verification
- Pay attention to trade offs in persisting to the chain. E.g. a hash for each message may be excessive.

3. **Verification Page**
   - Implement a web-based verification interface that allows a user to:
     - Input or paste original message content
     - Retrieve the on-chain hash and timestamp for a given transaction
     - Compare the computed hash of the provided content against the on-chain record
     - Display a clear pass/fail fidelity result with timestamp information
   - This page should be accessible independently of the messaging application
   - Reference example: Document Fidelity Proof — Upstream Exchange
4. **Understanding and Explanation**
   - Students must be able to explain: hash functions and why keccak256 is used, how Ethereum transactions work, gas costs, the immutability guarantees of blockchain, and the difference between on-chain and off-chain data

## Assessment Rubrics

Each minor is assessed using the rubric below. Marks are awarded on an individual basis during the interview.

**Grading Scale:**

| Level | Percentage Band |
|---|---|
| Excellent | 80–100% |
| Very Good | 60–79% |
| Good | 50–59% |
| Acceptable | 40–49% |
| Poor | 0–39% |

### Computer Networks & Cybersecurity Rubric (Mark Burkley) — 25% - 40 marks

| Criterion | Marks | Excellent (80–100%) | Very Good (60–79%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| **Network coding using sockets API** | 10 | *To be defined* | | | | |
| **Crypto coding using openSSL, webcrypto, etc. SSL certificate verification** | 10 | *To be defined* | | | | |
| **Secure** | 10 | *To be defined* | | | | |

| Criterion | Marks | Excellent (80–100%) | Very Good (60–79%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| **coding and input validation** | | | | | | |
| **Pentest and known vulnerabilities** | 10 | *To be defined* | | | | |

## C++ Programming Rubric (Kashif Memon) — 25% - 40 marks

| Criterion | Marks | Excellent (80–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| **C++ Component and Project Integration** | 10 | *Clear, working C++ component that is well connected to the messaging application and demonstrated successfully. Student explains its role clearly.* | Working C++ Working C++ component with a mostly clear connection to the project. Minor gaps in integration or demonstration | Basic working component connected to the project, but limited in scope or functionality. | Minimal or partly working component with weak connection to the project. | No meaningful C++ component, or the component does not work. |
| **Code Structure and Organisation** | 10 | *Code is well organised into appropriate .h/.hpp and .cpp files. Build instructions are clear, and CMake or another* | Mostly well organised across multiple files. Build instructions are present with minor omissions or clarity issues. | Some file organisation is used, but structure is basic, inconsistent, or only partly clear. | Limited organisation; code is difficult to navigate or mostly contained in one file without strong justification. | Poorly organised code with little evidence of structured C++ |

| Criterion | Marks | Excellent (80–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | *documented build method is used effectively*. | | | | development. |
| **Functions, Classes, and OOP Design** | 10 | *Strong use of functions and classes to model relevant system parts such as User, Message, Conversation, Client, or MessageStore. Good use of constructors, public/private access, and clear object-oriented design*. | Good use of functions and classes with mostly appropriate access control and constructors. Minor design weaknesses. | Some use of functions and classes, but design is basic, inconsistent, or not fully justified. | Limited use of functions/classes. Code is mostly procedural or difficult to explain as an object-oriented design. | Little or no meaningful use of functions, classes, or object-oriented programming. |
| **Modern C++, Memory Safety, Documentation, and Interview Understanding** | 10 | *Good use of STL containers/algorithms, references, const, and safe memory management. README clearly explains build/run instructions. Student strongly explains code* | Mostly safe code with reasonable STL and modern C++ use. Documentation is clear, and student explains most important design choices. | Some STL use and generally safe code. Documentation is basic, and student can explain the main parts of the component. | Minimal modern C++ use or unclear memory ownership. Documentation is weak, and student struggles to explain important parts. | Unsafe memory management, major pointer issues, little use of C++ features, no useful documentation, or |

| Criterion | Marks | Excellent (80–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | *organisation, class design, memory ownership, STL use, and any AI tool use.*. | | | | student cannot explain the submitted code. |

## Cryptography Rubric (Eoin O'Brien) — 25%

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| **Authenticated Encryption** | 5% | AEAD correctly chosen and used end-to-end; nonce strategy is principled and demonstrably collision-free; associated data used meaningfully where appropriate; ciphertext is opaque to the server and tampering is detected and rejected at the demo. Student | AEAD correctly chosen and used; nonce strategy is sound; server cannot read or undetectably modify ciphertext. Minor gaps in associated data use or in justification. | Standard AEAD used correctly for message payloads; nonces handled adequately; basic confidentiality and integrity hold. Some weaknesses in edge cases (e.g. error handling, replay). | AEAD present but with concerning patterns (e.g. unclear nonce strategy, AEAD used inconsistently, missing associated data where it matters). Confidentiality holds in the common case but the design is fragile. | Non-AEAD scheme, custom construction, MAC-then-Encrypt, nonce reuse, ECB, or other forbidden primitives in security-relevant roles. Confidentiality or integrity does not hold. |

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | demonstrates exceptional command of why each choice was made. | | | | |
| **Key Establishment & Sender Authentication** | 5% | HPKE Mode_Auth (or equivalent justified construction) correctly implemented; trust model clearly specified and defended; recipients can verify message origin; key publication and lookup designed thoughtfully against an active or compromised server. Forward secrecy properties (or their absence) acknowledged honestly. | HPKE Mode_Auth or equivalent correctly used; trust model stated and reasonable; sender authentication works. Minor weaknesses in trust-model justification or key publication. | Shared cryptographic state established without server access to it; sender authentication present; basic trust model (typically TOFU) implemented and stated. Some gaps in defending against active attackers. | Key establishment present but with weaknesses (e.g. server can MITM new conversations without detection, unclear sender authentication, trust model not defended). | Server can read shared secrets; sender authentication absent or trivially forgeable; or use of textbook RSA / hand-rolled key exchange. |

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| **Password & Key Derivation** | 5% | Memory-hard password hashing (Argon2id) with well-justified parameters, or PBKDF2-HMAC-SHA256 with FIPS justification and OWASP-compliant iteration count; HKDF used with explicit info strings achieving clear domain separation; local private keys encrypted at rest under a separately-derived key with appropriately tuned parameters. Full lifecycle considered. | Appropriate password hashing function with reasonable parameters; HKDF used with domain separation; at-rest protection of long-term keys present. Minor weaknesses in parameter choice or info-string design. | Standard password hashing used with defensible parameters; HKDF used for key derivation; long-term keys protected at rest in some form. Limited evidence of careful parameter selection. | Password hashing present but with weak or unjustified parameters; HKDF used but with collisions or missing domain separation; at-rest protection weak or absent. | Plaintext password storage, raw hashing (SHA-256 with no KDF), passwords used directly as keys, or long-term private keys stored unprotected. |
| **Design Document** | 5% | Threat model engages | Threat model addresses all four | Threat model present and | Document present but threat model is | No design document, or |

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | seriously with all four attacker classes; properties surviving server compromise stated explicitly and accurately; every primitive justified at parameter level with citations to RFCs or papers (section numbers where appropriate); construction diagrams cover registration through receive; limitations acknowledged honestly; document reads like a small security audit. | attacker classes adequately; most primitives justified with appropriate citations; diagrams present and clear; limitations stated. Minor gaps in rigour or citation discipline. | identifies the main attacker classes; primitives justified but mostly at the algorithm level rather than parameter level; diagrams adequate; some limitations stated. | shallow; justifications amount to "it's standard"; diagrams unclear or missing key flows; limitations not engaged with. | document fails to engage with threat model, justifications, or limitations. |
| **Understanding & Defence** | 5% | Articulates AEAD, nonce handling, | Strong understanding of core concepts; can | Adequate understanding; can describe what was | Limited understanding; can describe the | Cannot explain the cryptographic |

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | domain separation, KDF parameter selection, threat model, and trust model fluently; can explain consequences of nonce reuse and other failure modes precisely; defends every deviation from recommended primitives with substantive reasoning; engages critically with the design and identifies its own limitations unprompted. | explain most design decisions clearly; minor gaps on deeper questions (e.g. KDF parameter rationale, forward secrecy properties). | built and why at a high level; struggles with deeper or adversarial questions. | implementation but not justify it; significant gaps on threat model or primitive choice. | design; "I didn't do the implementation" or equivalent; no evidence of engagement with the material. |

## Blockchain Rubric (Andrew Le Gear) — 25%

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| **Smart Contract** | 5% | Correctly implemented Solidity contract deployed to | Contract deployed and functional; stores hashes | Contract deployed but with limited | Contract partially implemented or not deployed; significant | No contract or non-functional; |

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | Sepolia; stores hashes and timestamps; well-structured code with events and access controls | correctly; minor structural issues | functionality or minor bugs; basic hash storage works | issues with logic or deployment | fundamental misunderstanding of smart contracts |
| **Message Digest Integration** | 5% | Seamless integration; all messages hashed and recorded on-chain; transaction hashes stored and retrievable; handles errors gracefully | Most messages recorded on-chain; minor gaps in integration; transactions generally reliable | Basic integration working; some messages recorded but inconsistent; limited error handling | Minimal integration; few messages actually written to chain; significant reliability issues | No integration between messaging app and blockchain; or completely non-functional |
| **Verification Page** | 5% | Polished verification interface; correctly computes and compares hashes; clear pass/fail display with timestamp; works independently of main app | Verification page functional; compares hashes correctly; minor UI or usability issues | Basic verification works but with limitations; may require manual input of transaction details | Verification page exists but does not reliably verify; significant functional gaps | No verification page or completely non-functional |
| **Understanding** | 5% | Excellent explanation of hashing, Ethereum transactions, gas, immutability, and on-chain vs off-chain tradeoffs; can | Good understanding of core concepts; can explain most design decisions; minor gaps in deeper knowledge | Adequate understanding of basics; struggles with deeper questions on | Limited understanding; can describe what was built but not why; struggles with fundamental | Cannot explain the blockchain component; no evidence of |

| Criterion | Marks | Excellent (70–100%) | Very Good (60–69%) | Good (50–59%) | Acceptable (40–49%) | Poor (0–39%) |
|---|---|---|---|---|---|---|
| | | discuss design decisions fluently | | gas optimisation or security implications | concepts | understanding |

## Academic Integrity

This is a group project. While collaboration within teams is expected and encouraged, all submitted work must be the team's own. Use of AI tools is permitted and encouraged as a development aid, but students must:

- Understand and be able to explain all code in their submission
- Submit AI prompt artefacts as described above
- Be prepared to critically evaluate AI-generated code during the interview

Plagiarism, contract cheating, or submission of work that a student cannot explain will be treated as an academic integrity violation under University of Limerick regulations.

## Incomplete Submissions

Incomplete submissions or those missing any of the required elements may be penalised. Late submissions will be subject to standard University of Limerick late penalties. While this is a group project, grades are awarded individually. Lecturers reserve the right to adjust individual grades up or down based on each student's contribution and their demonstrated understanding during the interview.